

Module 12 - chapter 15

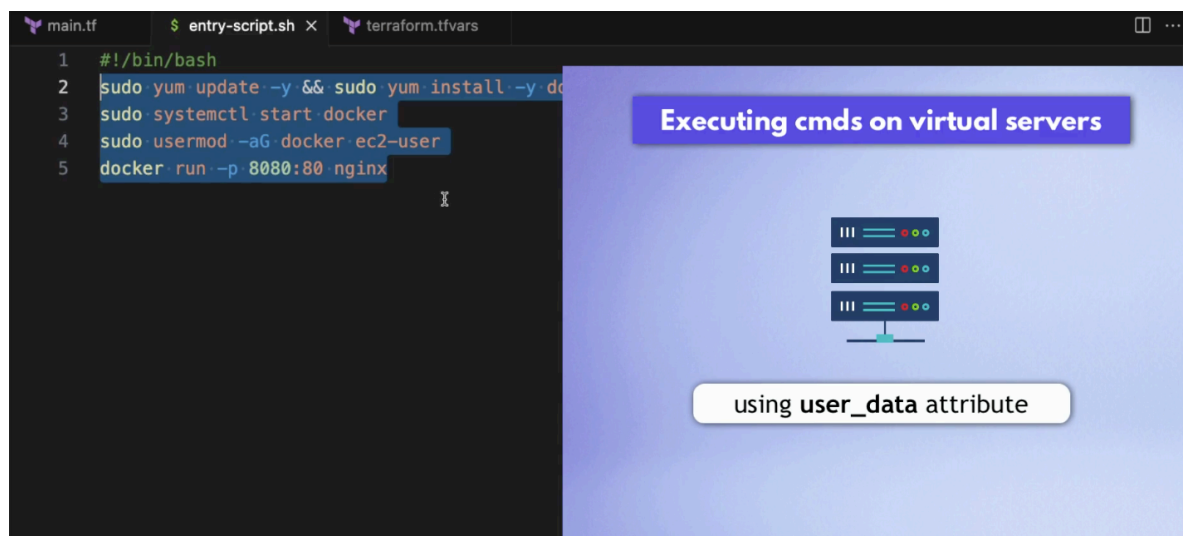
Provisioners in Terraform

The project files used in this lecture can be found here:

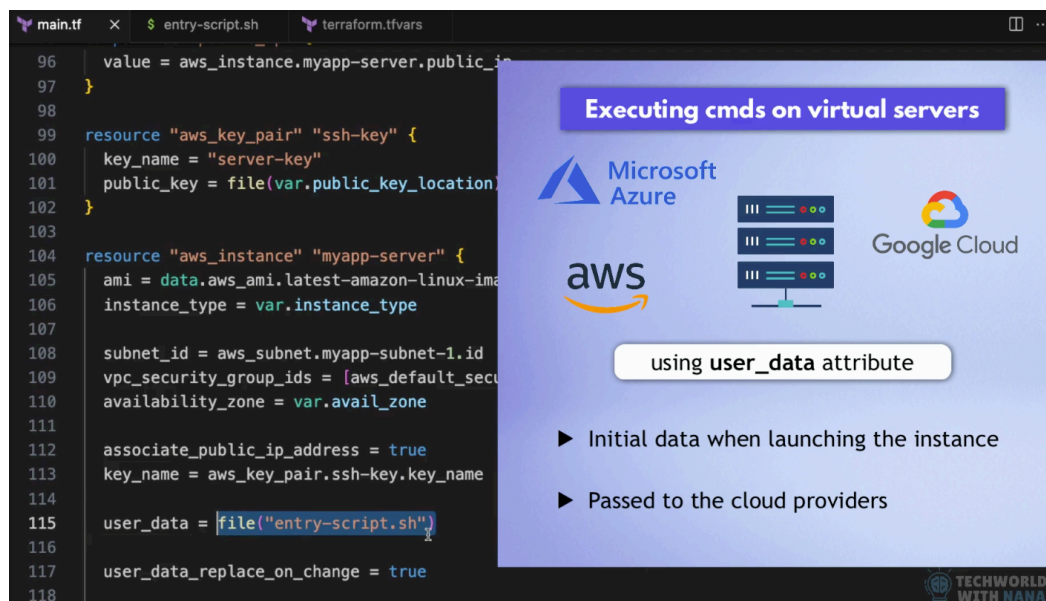
<https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/terraform-learn/-/tree/feature/provisioners>

Provisioners

Sometimes we need to execute commands in the provisioned servers like below by passing 'user_data'



AWS, Azure and GCP support this attribute 'user_data'

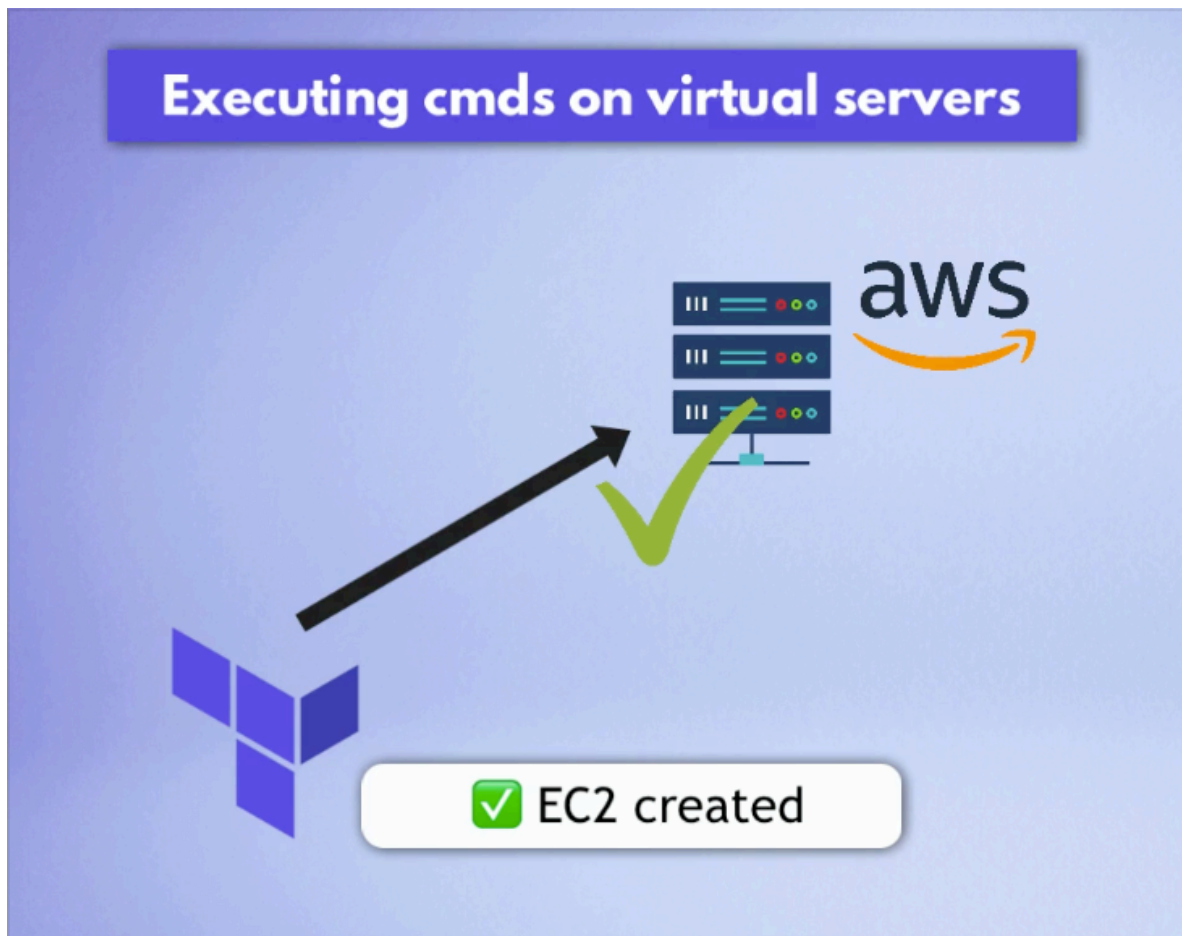


We need to be clear of the process here:

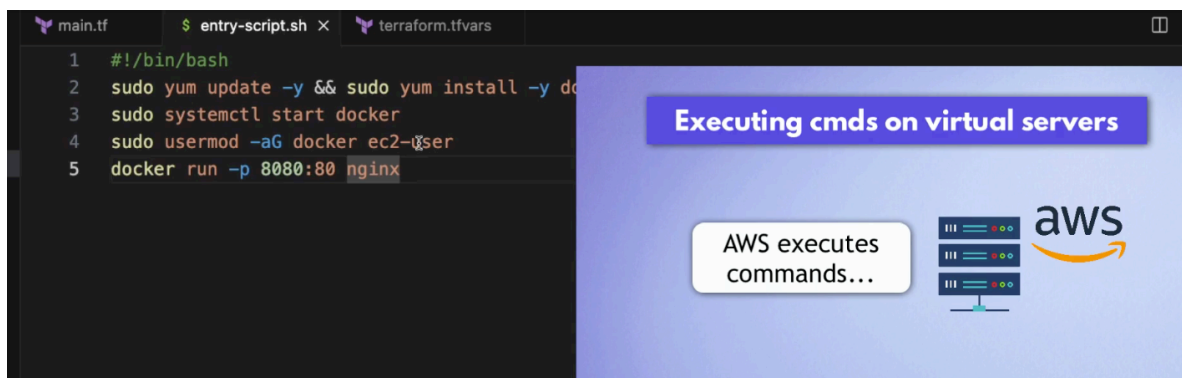
The commands will be handed over to the cloud provider. TF won't deal with

the commands.

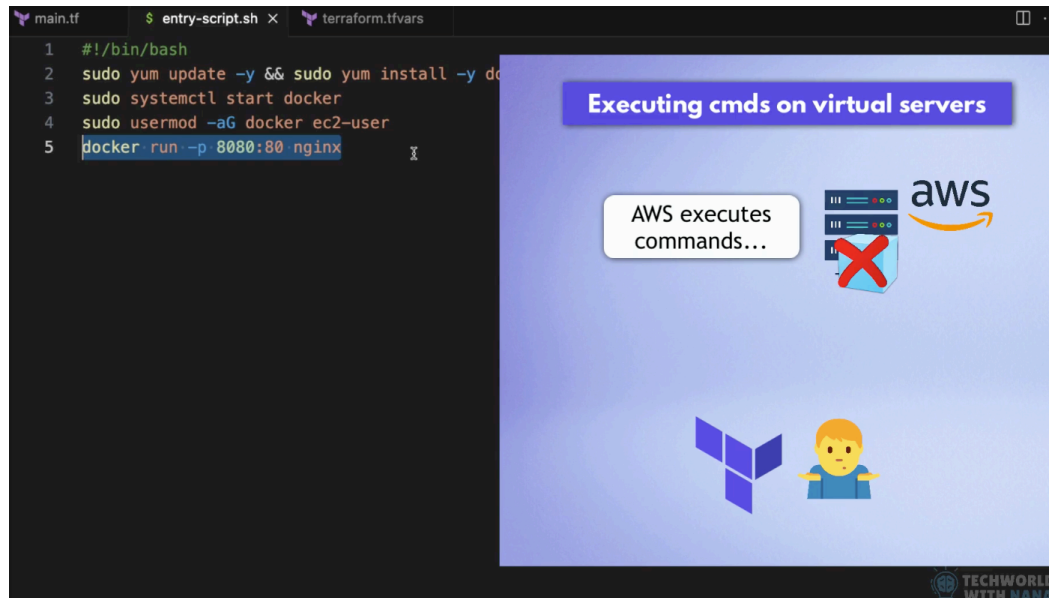
TF creates the EC2 instance and hands over the commands to AWS then TF job is done.



Then once the EC2 server gets initialised then AWS will execute the commands passed by TF



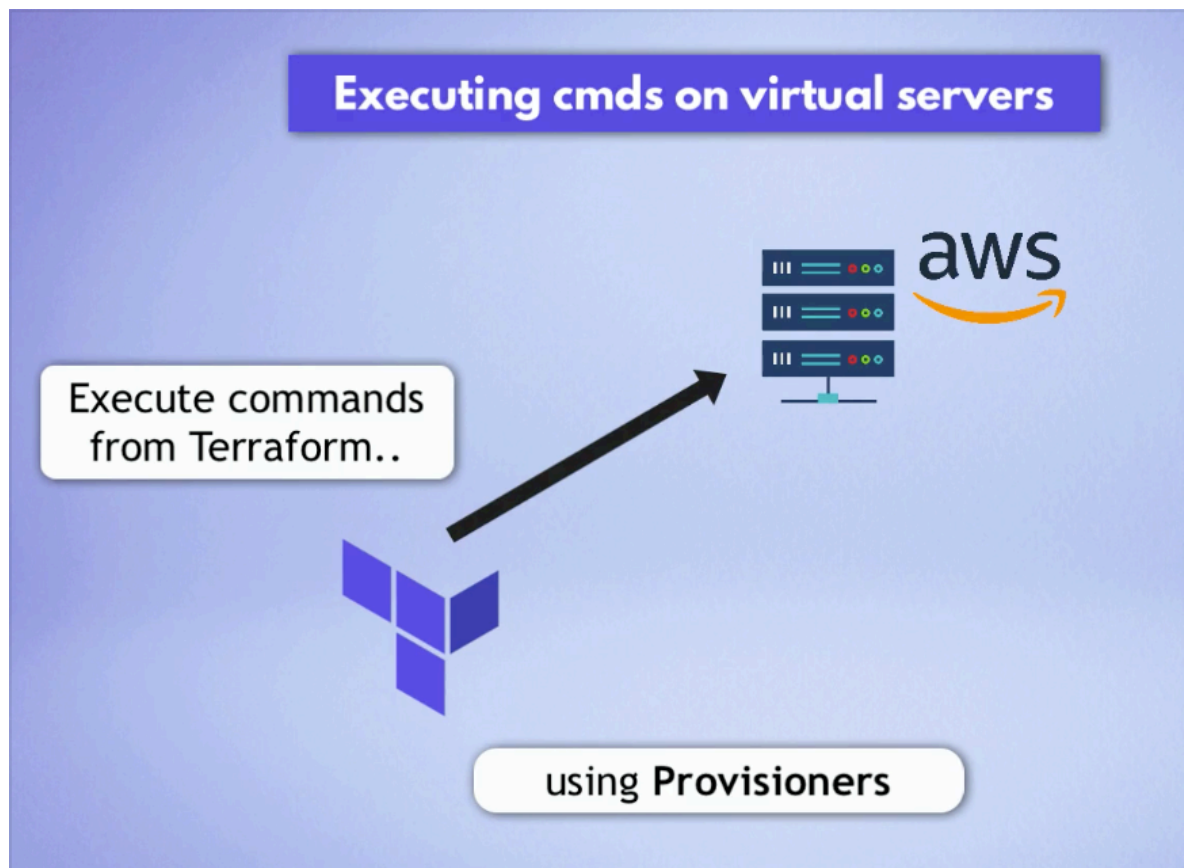
So if the commands fail AWS knows but TF doesn't



And TF won't log the error back to us regarding the commands run by AWS. We will have to ssh into the server and debug for ourselves to find the issue (in Module 12 - chapter 14 my script failed due to indentation and TF was not aware of it, so I had to debug manually to find the problem)

Alternative for TF -> provisioners

💡 There are ways to execute commands from terraform using -> **provisioners**



"remote-exec" is a provisioner that allows us to connect to a remote server and execute commands in that remote server

"remote-exec" provisioner

- invokes script on a remote resource after it is created

inline - list of commands

```
...cation)
" {
  nux-image_id
-1.id
lt_security_group.default-sg.id)
_name
```



```

138
139     user_data = file("entry-script.sh")
140
141     user_data_replace_on_change = true
142
143     provisioner "remote-exec" {
144         | You, Yesterday • Uncommitted changes
145     }
146
147     tags = {
148         Name = "${var.env_prefix}-server"
149     }
150 }

```

To define the commands we use the 'inline' attribute. We can add any command like define env vars, create a dir, install something:

```

    provisioner "remote-exec" {
        inline = [
            "export ENV=dev",
            "mkdir newdir",
        ]
    }

```

Because we are using 'remote-exec' we also need to tell TF how to connect to the remote server even if the provisioner is inside the resource block 'aws_instance' the provisioner doesn't connect to the server by default.

So we use the 'connection' attribute.

```

135
136     associate_public_ip_address = true
137     key_name = aws_key_pair.ssh-key.key_name
138
139     # user_data = file("entry-script.sh") // commented out to use provisioner instead
140
141     user_data_replace_on_change = true
142
143     connection {
144     }
145
146     provisioner "remote-exec" {
147     }
148
149     tags = {
150     Name = "${var.env_prefix}-server"
151     }
152 }
153 }
154 }

```

We need to provide the type of connection, there are two types:

- Ssh
- Win rm

The default is ssh.

We need to pass the host (server's IP address) and because we are inside the "aws_instance" block to reference the instance we can use "self" (like we do in python or in java with 'this')

So 'self' refers to the object "aws_instance" and then we access the attribute of the object -> public_ip

```

142
143     connection {
144         type = "ssh"
145         host = self.public_ip
146     }
147
148     provisioner "remote-exec" {
149     }
150 }
151 }

```

And we need to pass the username and private key to get authenticated.

```
terraform main.tf
140 # user_data = file("entry-script.sh") // commented out to use p
141
142 user_data_replace_on_change = true
143
144 connection {
145   type = "ssh"
146   host = self.public_ip
147   user = "ec2-user"
148   private_key = file(var.private_key_location) You, Moments
149 }
150
151 provisioner "remote-exec" {
152
153 }
154
```

```
terraform.tfvars
1 vpc_cidr_block = "10.0.0.0/16"
2 subnet_cidr_block = "10.0.10.0/24"
3 avail_zone = "eu-west-2a"
4 env_prefix = "dev"
5 my_ip = "140.228.47.252/32"
6 instance_type = "t2.micro"
7 public_key_location = "/Users/astrid/.ssh/id_ed25519.pub"
8 private_key_location = "/Users/astrid/.ssh/id_ed25519"
```

With the above TF ssh into the instance and then can execute the commands directly in the server:

```
main.tf M X $ entry-script.sh terraform.tfvars
112
113 associate_public_ip_address = true
114 key_name = aws_key_pair.ssh_key
115
116 # user_data = file("entry-script.sh")
117
118 user_data_replace_on_change = true
119
120 connection {
121   type = "ssh"
122   host = self.public_ip
123   user = "ec2-user"
124   private_key = file(var.private_key_location)
125 }
126
127 provisioner "remote-exec" {
128   inline = [
129     "export ENV=dev",
130     "mkdir newdir"
131   ]
132 }
133
134 tags = {
135   Name: "${var.env_prefix}-ec2"
136 }
137 }
```

Difference



user_data: passing data to AWS

remote-exec: connect via SSH using Terraform

TECHWORLD WITH NANA

-> terraform plan

(I destroyed the previous infra, so all the resources will be created again + the provisioner and its connection)

```
}  
  
Plan: 7 to add, 0 to change, 0 to destroy.
```

```
Changes to Outputs:
```

```
+ ec2_public_ip = (known after apply)
```

-> terraform apply

```
Error: remote-exec provisioner error  
  
with aws_instance.myapp-server,  
on main.tf line 152, in resource "aws_instance" "myapp-server":  
152:   provisioner "remote-exec" {  
  
Failed to parse ssh private key: ssh: this private key is passphrase protected  
  
→ terraform git:(Module_12/Terraform_chapter-15) ×
```

My private key is passphrase protected and Terraform's remote-exec provisioner doesn't support passphrase-protected SSH keys directly

AI:

You have two options:

Option 1 — Remove the passphrase from your key (simplest)

```
ssh-keygen -p -f ~/.ssh/your-key.pem
```

```
# Enter old passphrase, then leave new passphrase empty
```

Option 2 — Use ssh-agent (keeps the passphrase)

```
eval $(ssh-agent)
```

```
ssh-add ~/.ssh/your-key.pem
```

```
# Enter passphrase once
```

Then run terraform apply in the same terminal session. Terraform will use the agent automatically.

Option 1 is simpler for local dev. Option 2 is better if you need to keep the passphrase for security reasons.

So I overwrite the passphrase following option 1 🙏

```
> ssh-keygen -p -f ~/.ssh/id_ed25519
Enter old passphrase:
Key has comment 'astridcaballero@gmail.com'
Enter new passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved with the new passphrase.
```

-> terraform apply
(I destroyed the infra first)

```
aws_instance.myapp-server: Still creating... [00m30s elapsed]
aws_instance.myapp-server: Provisioning with 'remote-exec'...
aws_instance.myapp-server (remote-exec): Connecting to remote host via SSH...
aws_instance.myapp-server (remote-exec): Host: 18.175.239.68
aws_instance.myapp-server (remote-exec): User: ec2-user
aws_instance.myapp-server (remote-exec): Password: false
aws_instance.myapp-server (remote-exec): Private key: true
aws_instance.myapp-server (remote-exec): Certificate: false
aws_instance.myapp-server (remote-exec): SSH Agent: true
aws_instance.myapp-server (remote-exec): Checking Host Key: false
aws_instance.myapp-server (remote-exec): Target Platform: unix
aws_instance.myapp-server (remote-exec): Connected!
aws_instance.myapp-server: Creation complete after 33s [id=i-02014bd033d117e91]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

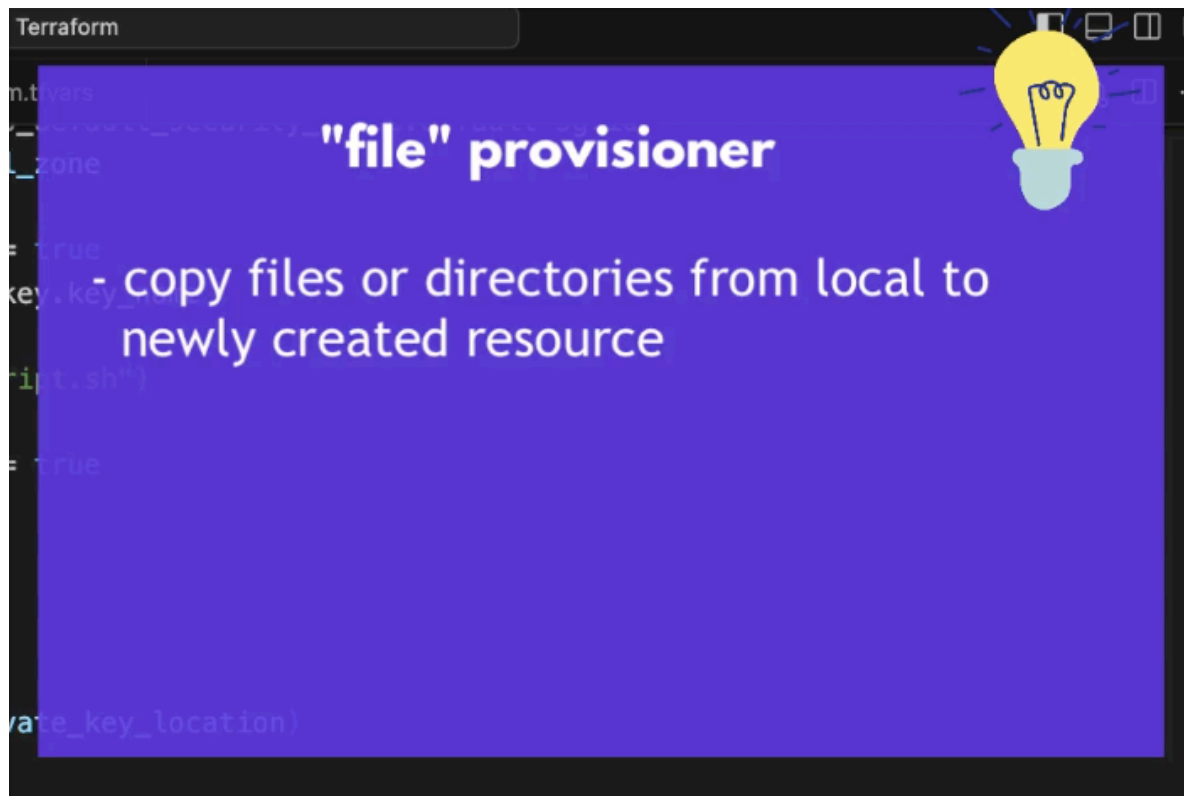
ec2_public_ip = "18.175.239.68"
→ terraform git:(Module_12/Terraform_chapter-15) ×
```

We can see that TF provisioned with 'remote-exec' and created the remote connection and that it made the connection 🙌

Now we ssh into our EC2 instance and check that newdir exists -> TF run this command line inside the EC2, see line 155 below

```
152     provisioner "remote-exec" {
153         inline = [
154             "export ENV=dev",
155             "mkdir newdir"
156         ]
157     }
```


created resource in this case our EC2.



And we need to pass the path on the local machine under the attribute 'source'

```
151
152     provisioner "file" {
153         source = "entry-script.sh"
154     }
155
```

Then we pass the 'destination' which is the path in the remote machine where we are copying the file

So we check the path in our EC2:

```
-> ssh ec2-user@18.175.239.68
-> pwd
```


If we want the file copy into another EC2 we can have the connection block inside the 'file' provisioner block

```
120 connection {
121   type = "ssh"
122   host = self.public_ip
123   user = "ec2-user"
124   private_key = file(var.private_key_location)
125 }
126
127 provisioner "file" {
128   source = "entry-script.sh"
129   destination = "/home/ec2-user/entry-script-on-ec2.sh"
130 }
131
132 provisioner "file" {
133   source = "entry-script.sh"
134   destination = "/home/ec2-user/entry-script-on-ec2.sh"
135 }
136 connection {
137   type = "ssh"
138   host = someotherserver.public_ip
139   user = "ec2-user"
140   private_key = file(var.private_key_location)
141 }
142 }
```

There is a cleaner way to pass the file and run it inside EC2 -> we use 'script' instead of 'inline'

```
150 }
151
152 # provisioner "file" {
153 #   source = "entry-script.sh"
154 #   destination = "/home/ec2-user/entry-script-on-ec2.sh"
155 # }
156 You, 7 minutes ago • Uncommitted changes
157 provisioner "remote-exec" {
158   script = ["entry-script.sh"]
159 }
160 }
```

And when we use 'script' attribute then we don't need the 'file' provisioner. And with 'script' attribute we pass the local file path, see line 158 🙏

There is a third provisioner -> 'local-exec'
We use this to execute commands locally (in our laptop)

"local-exec" provisioner



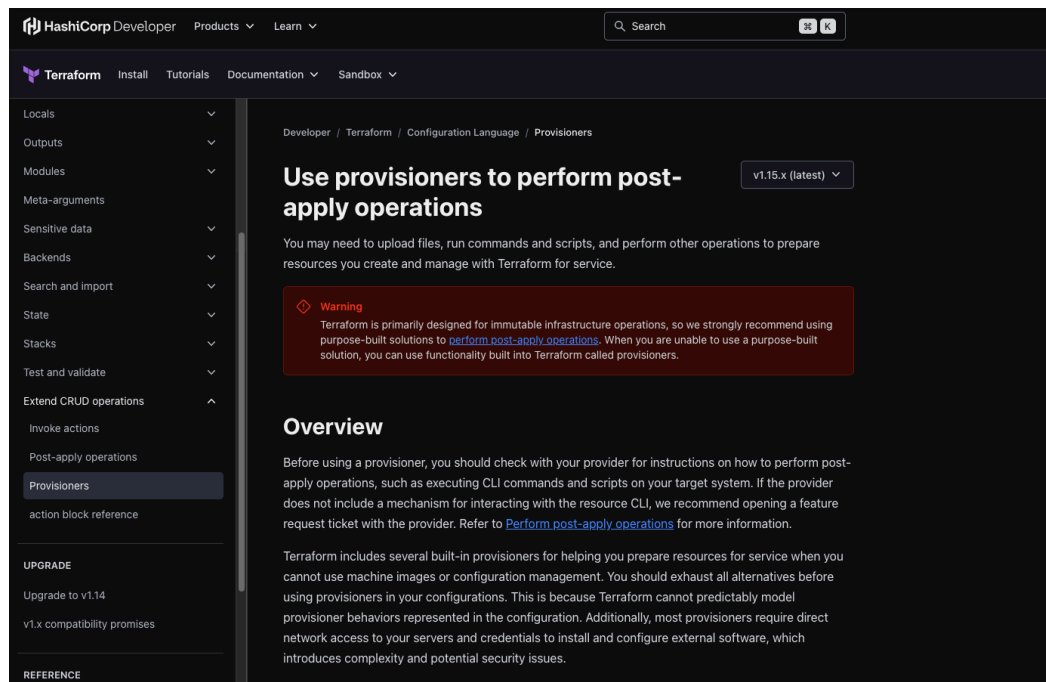
- invokes a local executable after a resource is created
- locally, NOT on the created resource!

For example we want to get the `public_ip` of the created EC2 instance and store it in a `.txt` file:

```
152 # provisioner "file" {
153 #   source = "entry-script.sh"
154 #   destination = "/home/ec2-user/entry-script-on-ec2.sh"
155 # }
156
157 provisioner "remote-exec" {
158   script = ["entry-script.sh"]
159 }
160
161 provisioner "local-exec" {
162   command = "echo ${self.public_ip} > ec2-public-ip.txt"
163 }
164
```

Provisioners are not recommended by TF !!

<https://developer.hashicorp.com/terraform/language/provisioners>



Provisioners don't always behave as expected.

'user_data' is better if the cloud provider makes it available

Provisioner -> breaks the concept of idempotency -> running the same input/config returns always the same output

But when TF runs scripts, TF doesn't know what it was executed in the script.

And if we change the script then TF doesn't know if the change deviates from the current state

raform
vars
stall -y docker

Provisioners are not recommended



- Use user_data if available
- Breaks idempotency concept
- TF doesn't know what you execute
- Breaks current-desired state comparison

And TF can't control how we write the scripts, they can not be idempotent for example and TF can't control that either.

Alternatives

If I want to run things on servers the use config management tools
-> ansible

Alternative to remote-exec

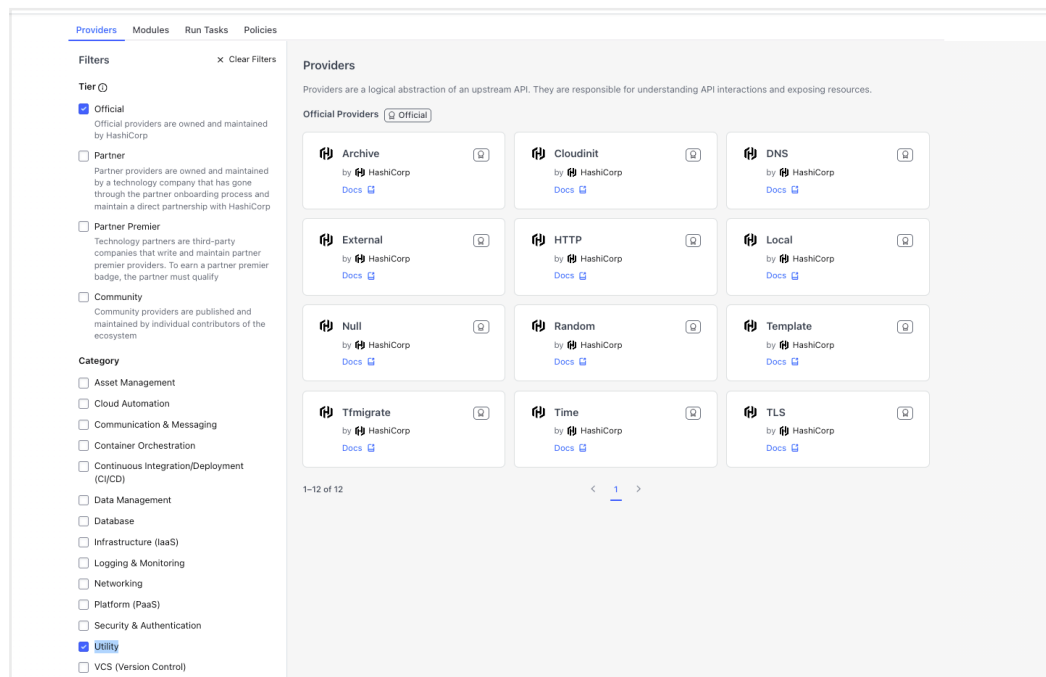
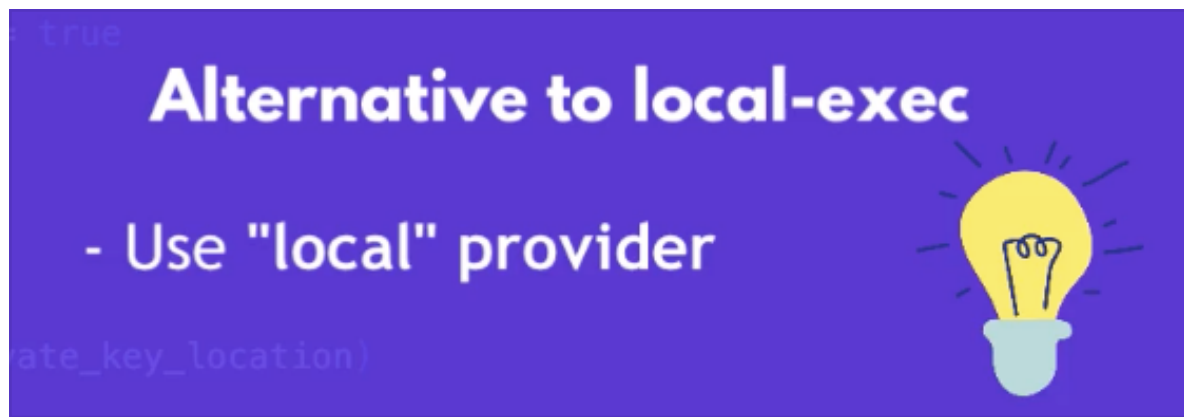
- Use configuration management tools
- Once server provisioned, hand over to those tools

puppet **ANSIBLE** **CHEF**

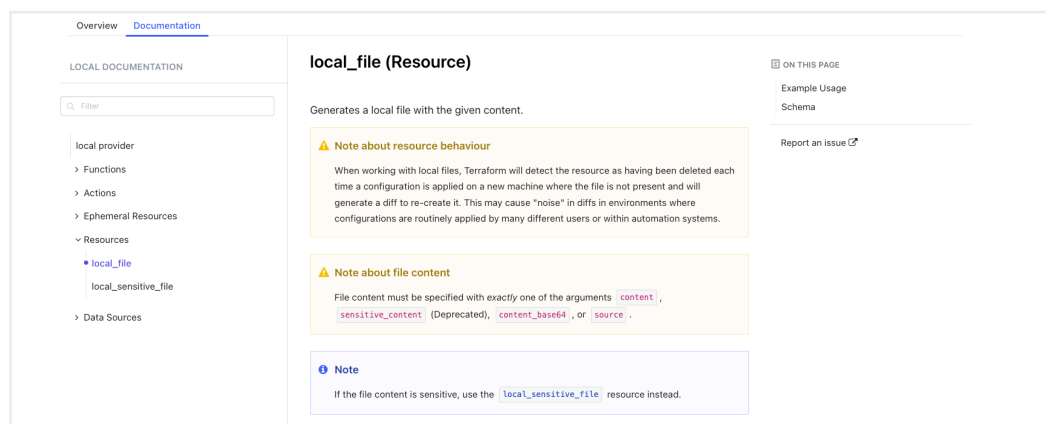
TECHWORLD WITH NANA

These config management tools have better visibility inside the VM -> they can compare the state and can manage and control how things get executed in the VM.

Regarding the local-exec there is a better way by using a provider called 'local Provider' which is maintained by HashiCorp



<https://registry.terraform.io/providers/hashicorp/local/latest/docs>



The provider has a way to compare the current-desire state so it can detect the changes.

Another alternative for 'remote-exec' is to execute the scripts separate from TF like using Jenkins

Alternative

- Execute scripts separate from Terraform
- From CI/CD tool



Provisioner failure

If we use 'remote-exec' and Tf can't find the file. -> TF will taint the EC2 instance, it will be marked for deletion.

So if we have this code

```
131
132   provisioner "remote-exec" {
133     script = "script.sh"
134   }
135
```

And execute -> terraform apply

```
Error: remote-exec provisioner error

  with aws_instance.myapp-server,
    on main.tf line 132, in resource "aws_instance" "myapp-server":
132:   provisioner "remote-exec" {

Failed to open script 'script.sh': open script.sh: no such file or directory

nana@macbook /Users/nana/terraform [master]
6
```

We get an error message because the file can't be found on the remote server

Then we check the AWS UI, the instance was created but we don't have a 'success' output.

Instances (1) Info							Refresh	Connect	Instance state ▾	Actions ▾	Launch
Find Instance by attribute or tag (case-sensitive)											
Instance state = running X							Clear filters				
☐	Name ✎	Instance ID	Instance state ▾	Instance type ▾	Status check	Alarm status					
☐	dev-server	i-0feeae90c1592af86	Running 🔍 🔍	t2.micro	🕒 Initializing	No alarms +					